Loan Navigator Agent Suite

Technical Project Documentation & Architecture Overview

1. Abstract

In India’s fast-paced fintech space, BlueLoans4all is empowering micro-entrepreneurs by offering accessible, small-ticket loans. Their support centers, however, face a deluge of repetitive yet vital queries like EMI status, prepayment scenarios, and top-up eligibility. This project introduces a multi-agent AI system, built using LangGraph, which powers the Agentic App acting as a smart "Loan Navigator". The solution combines NLP-to-SQL capabilities, RAG-based policy lookups from a vector database, and a what-if simulation engine to provide accurate, secure, and compliant answers to customer queries. By automating these interactions, the system enhances the borrower experience, reduces operational load, and ensures regulatory adherence.

2. Objective & Problem Statement

The core issue is the inefficiency and risk associated with manually resolving high-volume, low-complexity customer queries. Support staff spend valuable time fetching data from a loan database and cross-referencing policy documents to answer simple questions about EMIs, prepayments, and eligibility. This process is slow, error-prone, and prevents staff from focusing on more complex customer issues, creating a bottleneck that hinders the company's ability to scale efficiently.

To resolve this, we developed an autonomous AI agent suite that accurately resolves common loan queries, simulates financial scenarios, and ensures all responses are compliant and audit-ready. The goal is to automate the support process, providing 24x7 availability while significantly reducing manual overhead and improving key business metrics like query resolution time and prepayment uptake.

3. Technology Stack

Layer / Category	Technology / Service
Cloud Platform	Google Cloud Platform (GCP)
AI Service	Google Vertex AI (for Gemini Models) & Google GenAI Embeddings
Orchestration Framework	LangGraph
Vector Database	ChromaDB
Frontend UI	Streamlit
Deployment	Docker, Google Cloud Run, Google Artifact Registry
Observability	Google Cloud's operations suite (Logging & Monitoring), Langfuse
Security & Secrets	Google Secret Manager

4. Multi-Agent Architecture

The Loan Navigator Agent Suite is structured as an interactive state-based graph using LangGraph. A central "Supervisor" agent serves as the brains of the system, determining the user's intent and routing the execution state to specialized agent nodes.

Agent Breakdown

- **Supervisor Agent**

  Role: Central Orchestrator & Router.
  Logic: Uses Gemini with structured output mapping to classify the incoming query and set the destination field.

- **SQL Analyst Agent**

  Role: Relational Database Querier.
  Logic: Generates SQLite-compliant SQL queries using LangChain LCEL from schema descriptions. It queries the loan_data table within the database and handles fallback scenarios if no record matches are returned.

- **Policy Guru Agent**

  Role: Policy Compliance & Retrieval.
  Logic: Runs a Retrieval-Augmented Generation (RAG) pipeline. It utilizes Google Generative AI Embeddings to query a persistent Chroma vector database, identifying policy clauses from ingested compliance and underwriting PDFs.

- **What-If Calculator Agent**

Role: Numerical Financial Simulation.

Logic: Parses input parameters using an LLM JSON parser, and routes them to a native Python amortization engine that evaluates prepayment impacts.

5. Implementation & Code Base

State Definition (state.py)

The state dictionary for LangGraph execution keeps track of historical messages, categorized intent, SQL outputs, policy answers, and simulator results.

Supervisor Orchestrator (supervisor.py)

This is the core manager of the LangGraph state. It handles routing and maps structured JSON intents dynamically.

SQL Analyst Agent (sql_agent.py)

The database query module uses structured inputs and a secure executor connection to evaluate loan states without exposing the database to unwanted commands.

6. Observability & Tracing Architecture

The system is instrumented for dual-layer tracing, allowing infrastructure monitors and application designers to track operations end-to-end.

1. Langfuse Tracing

Used for tracing LLM execution graphs, parameter extractions, prompt templates, and semantic retrieval results. The CallbackHandler from langfuse.callback is registered with LangGraph's runtime configuration.

2. Google Cloud Operations Suite

Used for monitoring container health and tracking business performance metrics over time.

- **Structured Logging:** Standard logging streams directly to stdout/stderr inside the Docker container.
- **Custom Dashboard Metrics:** Captures real-time numerical signals and registers them against the global resource type in GCP Monitoring (agent invocation count and fallback count).

7. Cloud Deployment Runbook

Docker Container Configuration (Dockerfile)

This configures a lightweight Python 3.11 image, sets appropriate workspace paths, copies requirements, installs pip packages, mounts persistent database assets, and starts Streamlit on port 8080.

Steps for Deployment

1. Build the container image and upload to Artifact Registry.
2. Grant Secret Manager Accessor permissions to the default Cloud Run service account.
3. Deploy the container to Google Cloud Run, mapping secret references dynamically.